Table Contents

1.0 - Introduction.....3
2.0 - The Components.....3
2.1 Latch.....3
2.2 Decoder.....4
2.3 Finite State Machine (FSM).....6
3.0 - ALU Problem Set 1.....8
4.0 - ALU Problem Set 2.....11
5.0 - ALU Problem Set 3.....15
6.0 - Conclusion.....18

1.0 - Introduction

The purpose of this lab is to create an Arithmetic Logic Unit (ALU) that operates on two 8-bit inputs. The necessary components are an ALU, two latches, a 4-to-16 decoder, a Moore Finite State Machine (FSM), and seven-segment displays. The ALU performs operations using microcode signals provided by the decoder. The FSM cycles through its states to decide which operation the ALU should do. Each clock cycle, the latches store and forward input values to the ALU. Finally, the 7-segment displays will display your student number based on the FSM's current state and ALU output.

2.0 - The Components

2.1 Latch

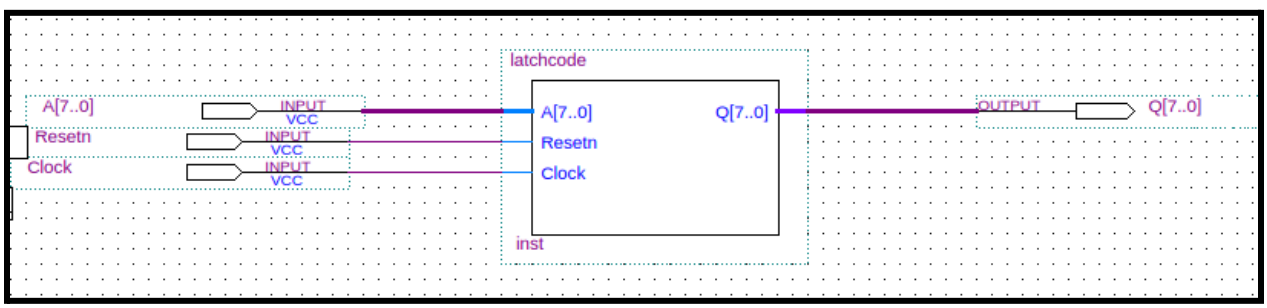
The latches act as an 8-bit register, briefly storing inputs A and B throughout each clock cycle. They record the input values on the clock's rising edge and use those stored values as outputs in the following cycle.

Reset	**Clock**	**D (Input)**	**Q (Output)**
0	d	ddddddd	00000000
1	Rising	00100000	00100000
1	Falling	00100000	00100000
1	Rising	00000010	00000010

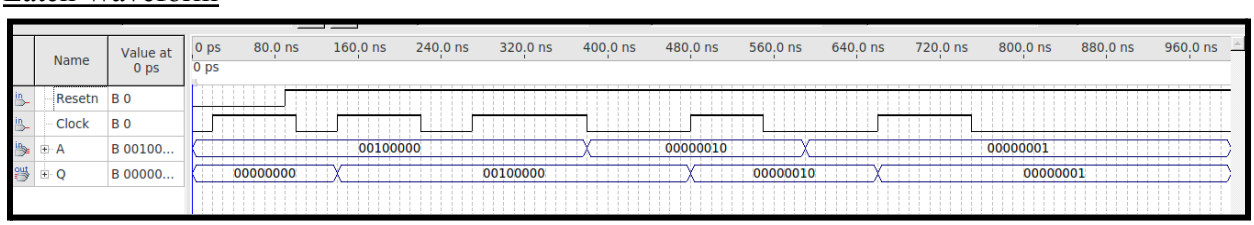
Latch Code

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

entity latchcode is
    PORT (
        A       : IN  STD_LOGIC_VECTOR(7 DOWNTO 0);  -- 8bit A input
        Resetn, Clock : IN STD_LOGIC;                -- 1 bit clk input and 1 bit reset input
        Q       : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)   -- 1bit output
    );
end latchcode;

ARCHITECTURE Behavior of latchcode is
begin
    process (Resetn, Clock)  -- process takes reset and clock as input
    begin
        if Resetn = '0' then
            Q <= "00000000";                          -- when reset is '0' the latch does not operate
        elsif (Clock'EVENT AND Clock = '1') then
            Q <= A;                                   -- level sensitive based on clock
        end if;
    end process;
end Behavior;
```

Latch Schematic



Latch Waveform



2.2 Decoder

The decoder receives a 4-bit input from the FSM indicating the current state and activates one of its 16 output lines to the ALU. It turns this 4-bit value into a 16-bit microcode signal, which the ALU uses to decide which operation to conduct.

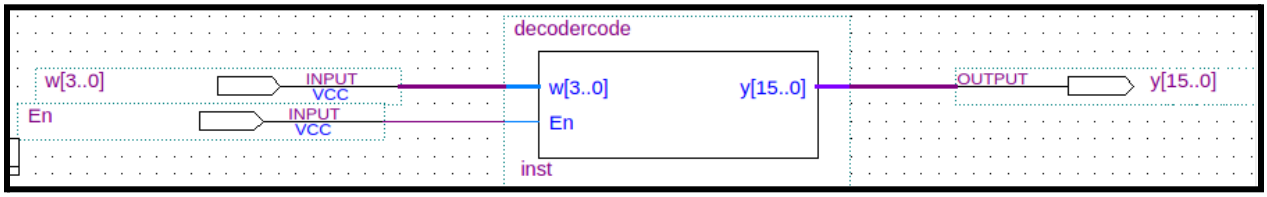
Decoder Truth Table

Enable (En)	Input (S - 4 bit)	Output (OP - 16 bit)
0	0000	0000000000000000
1	0000	0000000000000001
1	0001	0000000000000010
1	0010	0000000000000100
1	0011	0000000000001000
1	0100	0000000000010000
1	0101	0000000000100000
1	0110	0000000001000000
1	0111	0000000010000000
1	1000	0000000100000000

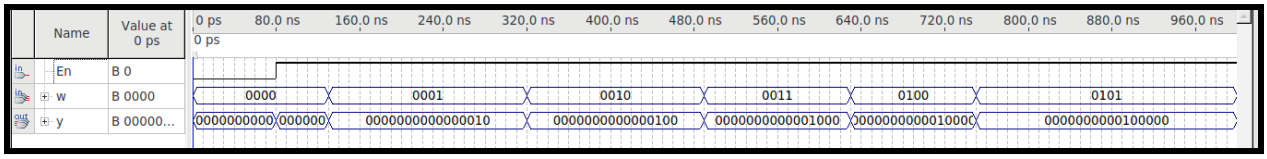
Decoder Code

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY decodercode IS
    PORT (
        w  : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
        En : IN  STD_LOGIC;
        -- changed from 0-15
        y  : OUT STD_LOGIC_VECTOR(15 DOWNTO 0)
    );
END decodercode;

ARCHITECTURE Behavior OF decodercode IS
    SIGNAL Enw : STD_LOGIC_VECTOR(4 DOWNTO 0);
BEGIN
    Enw <= En & w;

    WITH Enw SELECT
        y <=  "0000000000000001" WHEN "10000",   -- 0
              "0000000000000010" WHEN "10001",   -- 1
              "0000000000000100" WHEN "10010",   -- 2
              "0000000000001000" WHEN "10011",   -- 3
              "0000000000010000" WHEN "10100",   -- 4
              "0000000000100000" WHEN "10101",   -- 5
              "0000000001000000" WHEN "10110",   -- 6
              "0000000010000000" WHEN "10111",   -- 7
              "0000000100000000" WHEN "11000",   -- 8
              "0000000000000000" WHEN OTHERS;    -- default
END Behavior;
```

Decoder Schematic



Decoder Waveform



2.3 Finite State Machine (FSM)

The FSM cycles through nine states (S0–S8), updating its status on the rising edge of the clock. If data_in is 0 at the rising edge, the FSM remains in the current state; if data_in is 1, it moves to the next state. Each state generates a 4-bit value called current_state, which is passed to the 4-to-16 decoder.

Clock Cycle	**Present State**	**Next State**		**Output**	
		W = 0	W = 1	Current State	Student ID
1	0000	0000	0001	0000	0101
2	0001	0001	0010	0001	0000
3	0010	0010	0011	0010	0001
4	0011	0011	0100	0011	0011
5	0100	0100	0101	0100	0000
6	0101	0101	0110	0101	1001
7	0110	0110	0111	0110	0000
8	0111	0111	1000	0111	0100
9	1000	1000	0000	1000	1001

10	1001	dddd	dddd	dddd	dddd
11	1010	dddd	dddd	dddd	dddd
12	1011	dddd	dddd	dddd	dddd
13	1100	dddd	dddd	dddd	dddd
14	1101	dddd	dddd	dddd	dddd
15	1110	dddd	dddd	dddd	dddd
16	1111	dddd	dddd	dddd	dddd
1	0000	0000	0001	0000	dddd

FSM code

```
Library ieee;
use ieee.std_logic_1164.all;

entity FSMcode is
    port(
        clk           : in  std_logic;
        data_in       : in  std_logic;
        reset         : in  std_logic;
        student_id    : out std_logic_vector(3 downto 0);
        current_state : out std_logic_vector(3 downto 0)
    );
end entity;

architecture fsm of FSMcode is
    type state_type is (s0,s1,s2,s3,s4,s5,s6,s7,s8);
    signal yfsm : state_type;
begin

    process (clk, reset)
    begin
        if reset = '1' then
            yfsm <= s0;
            current_state <= "0000";
        elsif (clk'event AND clk = '1' AND data_in = '1') then
```

```
            case yfsm is
                when s0 => yfsm <= s1; current_state<= "0001"; -- 1
                when s1 => yfsm <= s2; current_state<= "0010"; -- 2
                when s2 => yfsm <= s3; current_state<= "0011"; -- 3
                when s3 => yfsm <= s4; current_state<= "0100"; -- 4
                when s4 => yfsm <= s5; current_state<= "0101"; -- 5
                when s5 => yfsm <= s6; current_state<= "0110"; -- 6
                when s6 => yfsm <= s7; current_state<= "0111"; -- 7
                when s7 => yfsm <= s8; current_state<= "1000"; -- 8
                when s8 => yfsm <= s0; current_state<= "0000"; -- 9
            end case;
        elsif (clk'event AND clk = '1' AND data_in = '0') then
            yfsm <= yfsm;
        end if;
    end process;

    process (yfsm, data_in)
    begin
        case yfsm is
            when s0 => student_id <= "0101"; -- 5
            when s1 => student_id <= "0000"; -- 0
            when s2 => student_id <= "0001"; -- 1
            when s3 => student_id <= "0011"; -- 3
            when s4 => student_id <= "0000"; -- 0
            when s5 => student_id <= "1001"; -- 9
            when s6 => student_id <= "0000"; -- 0
            when s7 => student_id <= "0100"; -- 4
            when s8 => student_id <= "1001"; -- 9
        end case;
    end process;

end fsm;
```

3.0 - ALU Problem Set 1

The previous components collaborated to organize the operations carried out by the ALU. Each FSM state activated a specified function. The main inputs were clock, reset, A, B, and OP. The clock controlled when the OP value was updated based on the FSM. The OP input came from the 4-to-16 decoder, which got its data from the FSM. The latches supplied the values for A and B, and the ALU performed the necessary operations using these inputs.

The ALU produced three outputs: "Neg," "R1," and "R2." Because some processes may give negative results, the negative signal was displayed as "Neg." The outputs R1 and R2

represented the final 8-bit result, which was separated into upper bits (R2) and lower bits (R1) and sent out accordingly.

<u>ALU Prob 1 Truth Table</u>

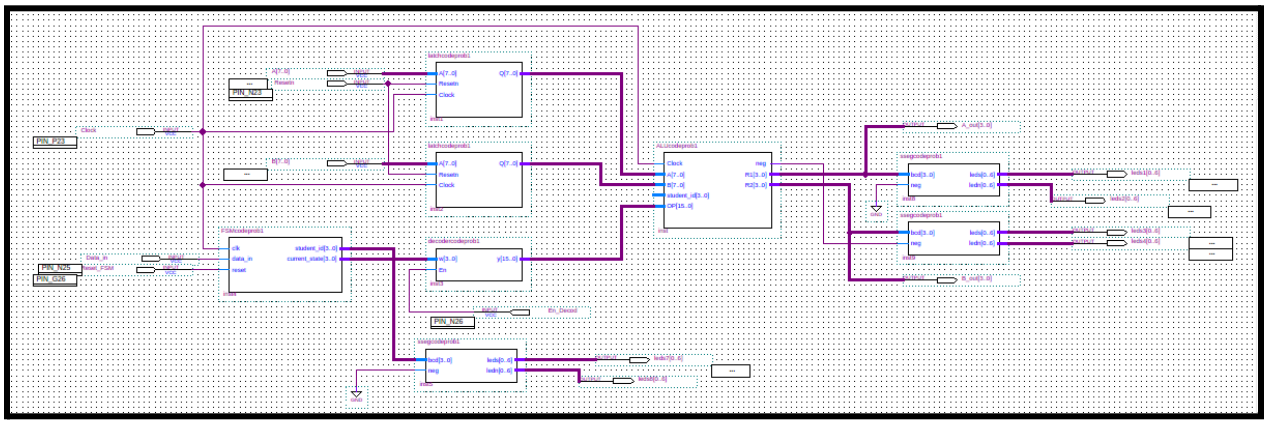
	DECODER		**ALU**		
Function Number	**Current State (4 bit)**	**Microcode (16 bit)**	**Operation**		**Expected Output** A: 1001 0000 B: 0100 1001
1	0000	0000000000000001	$sum(A,B)$	Sum	$1101\ 1001 \rightarrow (D9)_{16}$
2	0001	0000000000000010	$diff(A,B)$	Difference	$0100\ 0111 \rightarrow (47)_{16}$
3	0010	0000000000000100	$\overline{A}$	NOT A	$0110\ 1111 \rightarrow (6F)_{16}$
4	0011	0000000000001000	$\overline{A \cdot B}$	NAND	$1111\ 1111 \rightarrow (FF)_{16}$
5	0100	0000000000010000	$\overline{A + B}$	NOR	$0010\ 0110 \rightarrow (26)_{16}$
6	0101	0000000000100000	$A \cdot B$	AND	$0000\ 0000 \rightarrow (00)_{16}$
7	0110	0000000001000000	$A \oplus B$	XOR	$1101\ 1001 \rightarrow (D9)_{16}$
8	0111	0000000010000000	$A + B$	OR	$1101\ 1001 \rightarrow (D9)_{16}$
9	1000	0000000100000000	$\overline{A \oplus B}$	XNOR	$0010\ 0110 \rightarrow (26)_{16}$

ALU Prob 1 Code

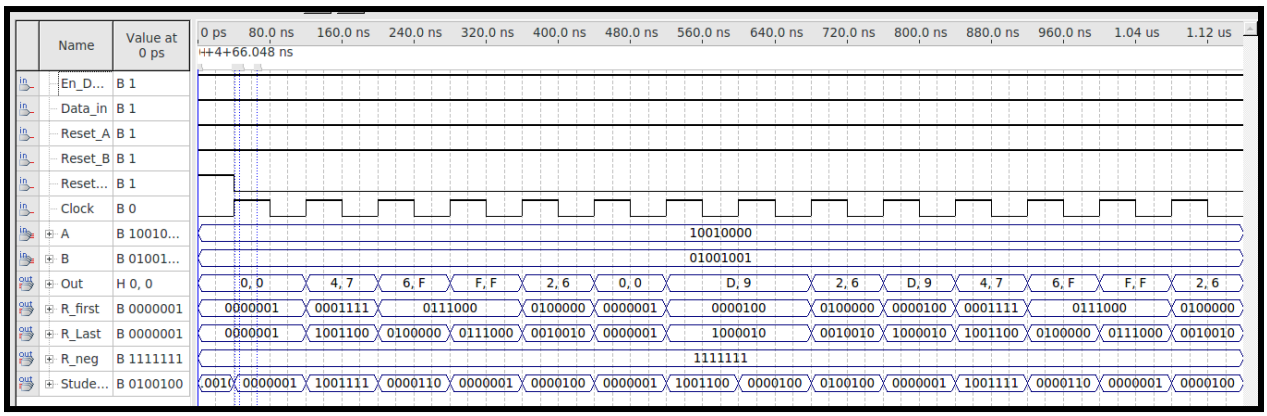
```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.std_logic_unsigned.all;
4   use ieee.numeric_std.all;
5
6   entity ALUcodeprob1 is
7       port ( Clock: in std_logic;                --clock input
8              A,B: in unsigned (7 downto 0);      --8 bit input from latch A and B
9              student_id: in unsigned (3 downto 0); --4 bit student id from FSM
10             OP: in unsigned(15 downto 0);       --16-bit selector for Operation from Decoder
11             neg: out std_logic;                 --is the result negative? set -ve bit output
12             R1: out unsigned(3 downto 0);       --lower 4-bits of 8-bit Results Output
13             R2: out unsigned(3 downto 0) );     --higher 4-bits of 8-bits Results Output
14  end ALUcodeprob1;
15
16  architecture calculation of ALUcodeprob1 is   --temp signal declaration
17      signal Reg1,Reg2,Result: unsigned(7 downto 0):=(others=> '0');
18      signal Reg4: unsigned(0 to 7);
19  begin
20      Reg1 <= A;     --temp store A in Reg1
21      Reg2 <= B;     --temp store B in Reg2
22
23  process(Clock,OP)
24  begin
25      if(rising_edge(Clock)) THEN
26          case OP is
27
```

```
28                  --addition --works
29                  WHEN "0000000000000001"   => Result  <= Reg1+Reg2; neg<='0';
30
31                  --subtraction --works --fix 7-seg neg number from lab3
32                  WHEN "0000000000000010"   => Result  <= Reg1-Reg2;
33                      if (Reg2>Reg1) then neg <= '1';    --if second number higher then its negative
34                      else neg <= '0';
35                      end if;
36
37                  --compliment
38                  WHEN "0000000000000100"   => Result  <= not (Reg1); neg<='0';
39
40                  --nand
41                  WHEN "0000000000001000"   => Result  <= (Reg1 nand Reg2); neg<='0';
42
43                  --nor
44                  WHEN "0000000000010000"   => Result  <= (Reg1 nor Reg2); neg<='0';
45
46                  --and
47                  WHEN "0000000000100000"   => Result  <= (Reg1 and Reg2); neg<='0';
48
49                  --xor
50                  WHEN "0000000001000000"   => Result  <= (Reg1 xor Reg2); neg<='0';
51
52                  --or
53                  WHEN "0000000010000000"   => Result  <= (Reg1 or Reg2); neg<='0';
54
55                  --xnor
56                  WHEN "0000000100000000"   => Result  <= (Reg1 xnor Reg2); neg<='0';
57
58                  WHEN OTHERS  => Result  <= "00000000" ; neg<='0';
59              end case;
60          end if;
61      end process;
62      R1 <= Result (3 downto 0);
63      R2 <= Result (7 downto 4);
64      end calculation ;
65
```

ALU Prob 1 Schematic



ALU Prob 1 Waveform



4.0 - ALU Problem Set 2

The ALU in problem 2 operates on inputs A and B using the decoder's microcode. Students can choose the ALU operation to perform. Possible operations include shift right, subtract, maximum, swap, increment, AND, bit-swap between inputs, rotate, and null. Each function is initiated by a distinct 16-bit microcode—for example, 0000000000000001 triggers the shift-right action. The ALU code is constructed with case statements that map each microcode to its corresponding operation. The inputs A and B are obtained by converting the student ID's last four digits from HEX to binary.

ALU Prob 2 Truth Table

Enable	Input (FSM State)	Microcode	Operation	Result (8 bits)	A OUT	B OUT
1	0000	0000000000000001	Shift A to right by 2 bits	11100100	1110	0100
1	0001	0000000000000010	(diff(A,B)) + 4	01001011	0100	1011
1	0010	0000000000000100	Max (A,B)	10010000	1001	0000
1	0011	0000000000001000	Swap upper 4 bit of A with lower 4 bit of B	10010000	1001	0000
1	0100	0000000000010000	A + 1	10010001	1001	0001
1	0101	0000000000100000	A • B	00000000	0000	0000
1	0110	0000000001000000	$\bar{A}$ (only upper 4 bits)	01100000	0110	0000
1	0111	0000000010000000	Rotate B to left by 3 bits	01001010	0100	1010
1	1000	0000000100000000	Null	00000000	0000	0000

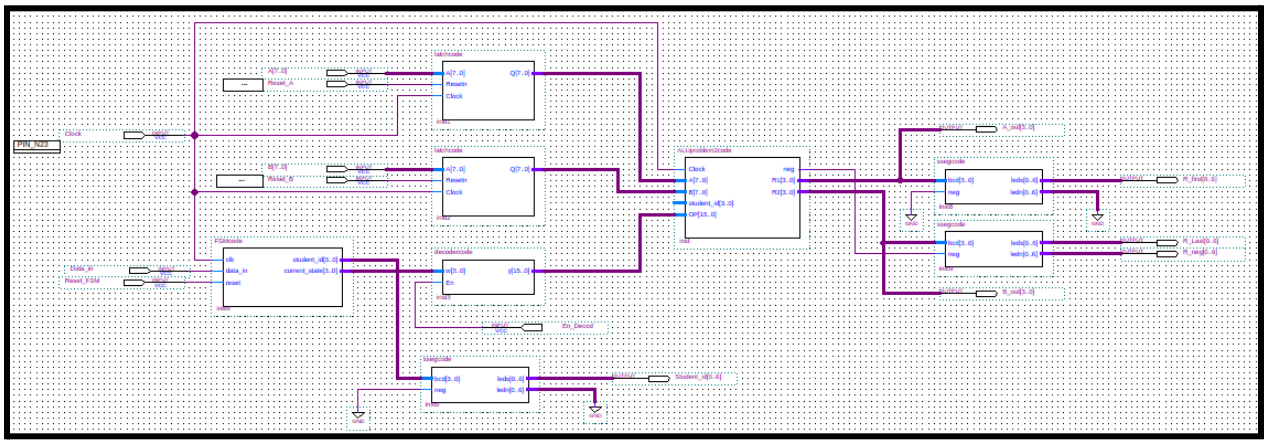
ALU Prob 2 Code

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;

entity ALUproblem2code is
    port ( Clock: in std_logic;                --clock input
           A,B: in unsigned (7 downto 0);      --8 bit input from latch A and B
           student_id: in unsigned (3 downto 0); --4 bit student id from FSM
           OP: in unsigned(15 downto 0);       --16-bit selector for Operation from Decoder
           neg: out std_logic;                 --is the result negative? set -ve bit output
           R1: out unsigned(3 downto 0);       --lower 4-bits of 8-bit Results Output
           R2: out unsigned(3 downto 0) );     --higher 4-bits of 8-bits Results Output
end ALUproblem2code;

architecture calculation of ALUproblem2code is   --temp signal declaration
    signal Reg1,Reg2,Result: unsigned(7 downto 0):=(others=> '0');
    signal Reg4: unsigned(0 to 7);
begin
    Reg1 <= A;     --temp store A in Reg1
    Reg2 <= B;     --temp store B in Reg2

process(Clock,OP)
begin
    if(rising_edge(Clock)) THEN

case OP is

--1
    WHEN "0000000000000001" =>
        Result(0) <= Reg1(2);
        Result(1) <= Reg1(3);
        Result(2) <= Reg1(4);
        Result(3) <= Reg1(5);
        Result(4) <= Reg1(6);
        Result(5) <= Reg1(7);
        Result(6) <= '1';
        Result(7) <= '1';
        neg <= '0';

--2
    WHEN "0000000000000010" => Result <= ((Reg1 - Reg2) + "0100");
        if (Reg2 > Reg1) then
            neg <= '1';  --if second number higher then its negative
        else
            neg <= '0';
        end if;

--3
    WHEN "0000000000000100" =>
        if (Reg1 > Reg2) then
            Result <= Reg1;
        else
            Result <= Reg2;
        end if;
        neg <= '0';
```

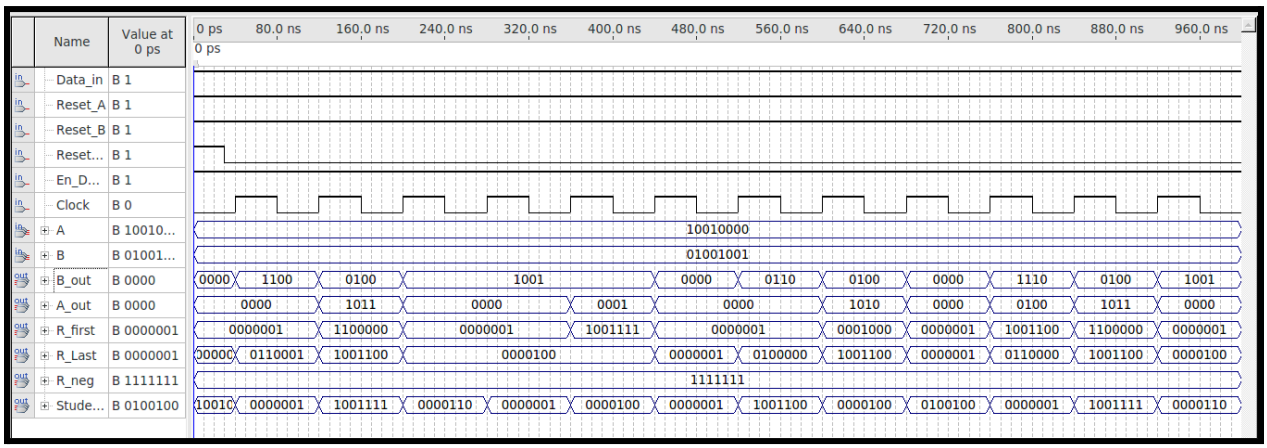
```
--4 --swap top 4 bits of A with lower 4 bits of B
    WHEN "0000000000001000"=>
        Result(0) <= Reg1(0);
        Result(1) <= Reg1(1);
        Result(2) <= Reg1(2);
        Result(3) <= Reg1(3);
        Result(4) <= Reg2(0);
        Result(5) <= Reg2(1);
        Result(6) <= Reg2(2);
        Result(7) <= Reg2(3);
        neg <= '0';

--5 --increase A by 1
    WHEN "0000000000010000"=>
        Result <= Reg1 + "00000001";
        neg <= '0';

--6 --and
    WHEN "0000000000100000"=>
        Result <= (Reg1 and Reg2);
        neg <= '0';

--7
    WHEN "0000000001000000"=>
        Result(0) <= Reg1(0);
        Result(1) <= Reg1(1);
        Result(2) <= Reg1(2);
        Result(3) <= Reg1(3);
        Result(4) <= not(Reg1(4));
        Result(5) <= not(Reg1(5));
        Result(6) <= not(Reg1(6));
        Result(7) <= not(Reg1(7));
        neg <= '0';
```

```
--8
    WHEN "0000000010000000"=>
        Result(7) <= Reg2(4);
        Result(6) <= Reg2(3);
        Result(5) <= Reg2(2);
        Result(4) <= Reg2(1);
        Result(3) <= Reg2(0);
        Result(2) <= Reg2(7);
        Result(1) <= Reg2(6);
        Result(0) <= Reg2(5);
        neg <= '0';

--9 --confirm
    WHEN "0000000100000000"=>
        Result <= "00000000";
        neg <= '0';

    WHEN OTHERS =>
        Result <= "00000000";
        neg <= '0';

end case;
end if;
end process;

R1  <= Result(3 downto 0); --since output 7-seg can only 4-bits
R2 <= Result(7 downto 4); --split into two 4 bits

end calculation;
```

ALU Prob 2 Schematic



ALU Prob 2 Waveform



5.0 - ALU Problem Set 3

For ALU problem set 3, it found out that an ALU was not necessary. The task only required showing "y" for odd numbers and "n" for even values. To accomplish this, the FSM and a seven-segment display (SSEG) were utilized. The SSEG display generally accepts a 4-bit input and converts it to a hexadecimal output displayed via LEDs. However, in this scenario, the display required to be changed to show "y" or "n" based on whether the student number digits were odd or even. This alteration is shown in the block diagram, waveform, and data table below.

ALU Prob 3 Truth Table

Student Number	Odd (y) (100100) or Even (n) (1101010)
5	y
0	n
1	y
3	y
0	n
9	y
0	n
4	n
9	y

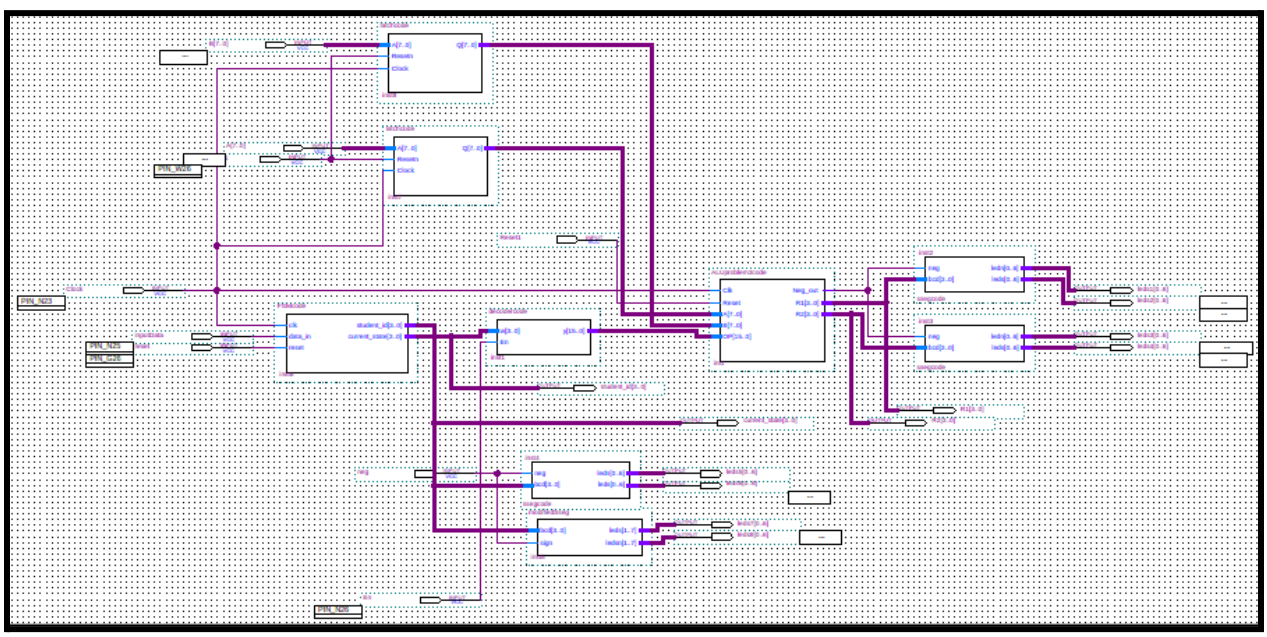
ALU Prob 3 Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity ALUproblem3code is
port (
        Clk, Reset      : in  std_logic;                        -- input clock signal
        A, B     : in  unsigned(7 downto 0);                -- 8-bit inputs from latches A and B
        OP       : in  unsigned(15 downto 0);               -- 16-bit selector for Operation from Decoder
        Neg_out  : out std_logic;                           -- is the result negative? Set -ve bit output
        R1, R2 : out unsigned(3 downto 0)
    );
end ALUproblem3code;

architecture calculation of ALUproblem3code is
    -- temporary signal declarations
    signal Reg1, Reg2, Result: unsigned(7 downto 0) := (others => '0');
    signal Reg4 : unsigned(0 to 7); -- unused, reserved
begin
    -- store inputs locally
    Reg1 <= A;  -- temporarily store A in Reg1 local variable
    Reg2 <= B;  -- temporarily store B in Reg2 local variable
    process(Clk, OP)
    begin
        if rising_edge(Clk) then -- Do the calculation @ positive edge of clock cycle.
            -- Perform operation depe Result <= (7 DOWNTO 0);nding on OP code
            case OP is
```

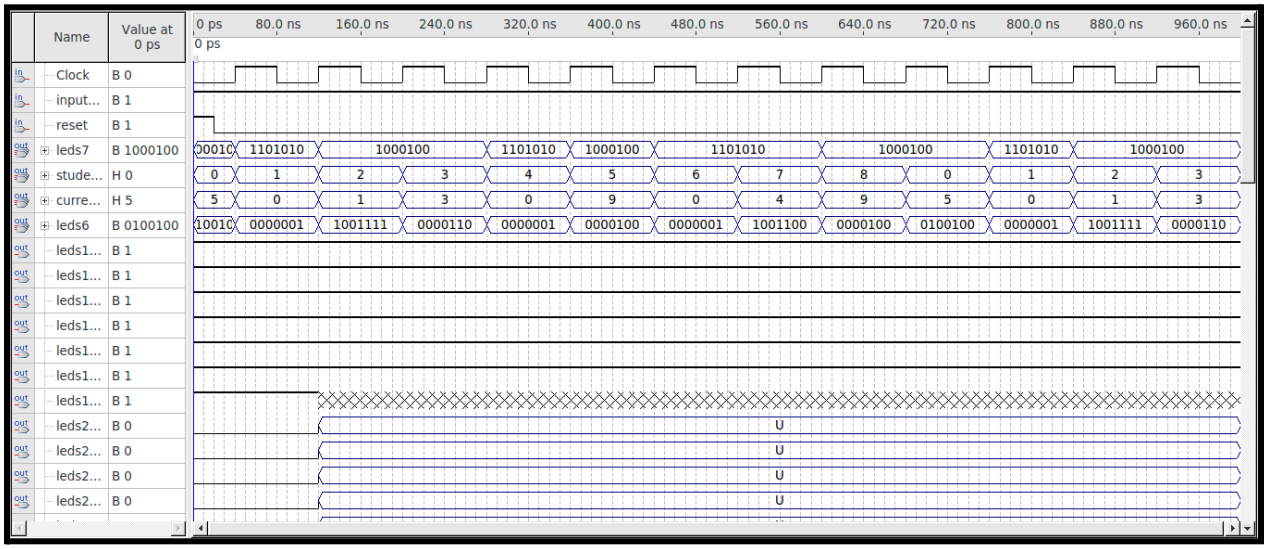
```
                when "0000000000000001" =>  -- Do Addition
                    Result <= Reg1 + Reg2;
                    Neg_out <= '0';
                when "0000000000000010" =>  -- Do Subtraction
                    --Result <= Reg1 - Reg2;
                    -- check if last bit is 1, if so, then do 2s complement, else then just print output
                    --Result <= not Reg2 + Reg1; -- 2s complement
                    Result <= Reg1 + NOT(Reg2) + 1;
                    if Reg1 < Reg2 then
                        Neg_out <= '1';
                    else
                        Neg_out <= '0';
                    end if;
                when "0000000000000100" =>  -- Do Inverse
                    Result <= not Reg1;
                    Neg_out <= '0';
                when "0000000000001000" =>  -- Do Boolean NAND
                    Result <= not (Reg1 and Reg2);
                    Neg_out <= '0';
                     -- Result <= ult <= Reg1 NAND Reg2
                when "0000000000010000" =>  -- Do Boolean NOR
                    Result <= not (Reg1 or Reg2);
                    Neg_out <= '0';
                     -- Result <= Reg1 NOR Reg2
                when "0000000000100000" =>  -- Do Boolean AND
                    Result <= Reg1 and Reg2;
                    Neg_out <= '0';
```

```
                when "0000000001000000" =>  -- Do Boolean XOR
                    Result <= Reg1 xor Reg2;
                    Neg_out <= '0';
                when "0000000010000000" =>  -- Do Boolean OR
                    Result <= Reg1 or Reg2;
                    Neg_out <= '0';
                when "0000000100000000" =>  -- Do Boolean XNOR
                    Result <= Reg1 xnor Reg2;
                    Neg_out <= '0';
                when others =>
                    Result <= (others => '0');  -- default: no operation -- Don't care, do nothing
            end case;
        end if;
    end process;
    R1 <= Result(3 downto 0);
    R2 <= Result(7 downto 4);
end calculation;
```

ALU Prob 3 Schematic



ALU Prob 3 Waveform



6.0 - Conclusion

This project was developed in Quartus utilizing a VHDL symbol-based design. The ALU created for issues 1, 2, and 3 may execute a variety of actions on inputs A and B using microcode. The system runs on a single clock cycle: the latches transmit inputs to the ALU, the FSM selects the microcode generated by the decoder, and the ALU conducts the operation. The results are then displayed on seven-segment monitors. Issue 1 involved general operations, issue 2 dealt with table-assigned operations, while problem 3 required checking if student number digits were odd or even.